

Project Exercise

Challenges when Designing Secure Web Applications

The OWASP has been reporting common vulnerabilities over almost two decades and many of them have remained the same, even though these problems are often relatively easy to cope with and have been solved in numerous ways. One of the reasons is that web applications often utilize (legacy) custom implementations of authentication and protection procedures, which include an input validation and a login form in the frontend, and some user database and password hashing and matching in the backend.

While this may reduce initial implementation costs due to less complexity and dependencies on third-party libraries or frameworks, it is rather naive to expect to attain reasonable and robust security with these self-defined mechanisms.

OAuth 2.0 and LRZ GitLab – Outsourcing to the Rescue

Rather than building our own authentication and authorization procedures in the OnlineIDE, we want to make use of the clearly specified and battle-proofed authorization framework OAuth 2.0. It defines a mechanism, which is provided by a trustworthy third party – in our case the LRZ GitLab – that we can conveniently rely on for authentication of users. This way we do not need to store user details ourselves, nor do we require any boilerplate code for e.g. password resetting, single-sign on (SSO) or two-factor authentication. The high-level target process is depicted in Fig. 4.

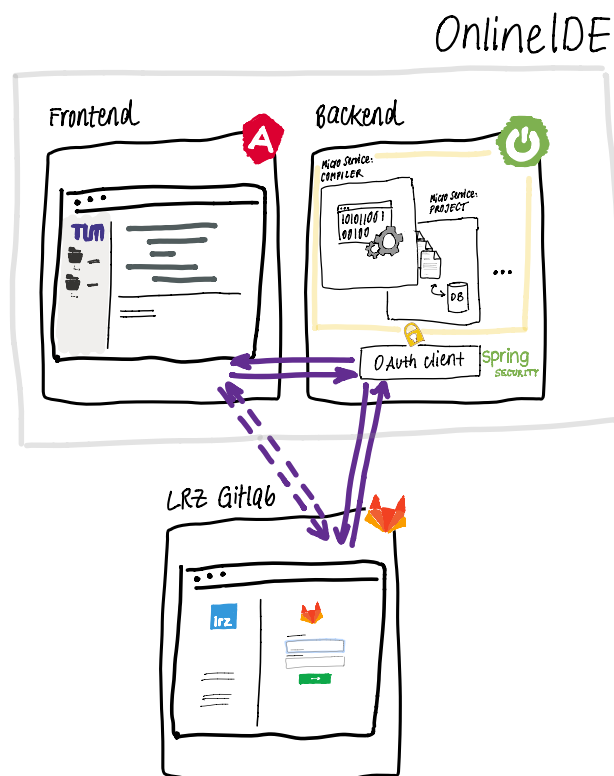


Figure 4: An overview of the OnlineIDE's integration with LRZ GitLab via OAuth 2.0

OAuth 2.0 is complex to implement and if you are not especially experienced in implementing network security protocols, your implementation might suffer some serious, but unobvious vulnerabilities¹¹. Therefore, it is advisable to make use of one of the many open-source libraries or frameworks that implement the OAuth 2.0 specification and have been used and reviewed by many fellow developers. In our case, this will be the *Spring Security framework* introduced in the last exercise.

¹¹Microsoft OAuth Vulnerability: <http://andrisatteka.blogspot.com/2014/09/how-microsoft-is-giving-your-data-to.html>

The outline of the following exercises is as follows: First, we will integrate OAuth 2.0 and GitLab with the Spring Security setup (see *OAuth client* in overview figure), we created on the previous exercise sheet (9). Then, we will implement the whole login and authentication process in the frontend, the Angular Application, which we started to build on exercise sheet 7.

Important: The whole Spring Security logic that you are building in these exercises will eventually be part of the *api-gateway* service. This will ensure that each request that is arriving in the backend (i.e., the microservices) of the OnlineIDE will be handled by the API gateway upfront. The API gateway itself will be introduced on exercise sheet twelve.

Exercise 4: Integrating OAuth 2.0 and GitLab with the OnlineIDE

This exercise will lead you through the process of integrating the OAuth 2.0 endpoints provided by the LRZ GitLab with a bare-bones Spring Boot application using Spring Security. As a result, we do not have to implement anything of the OAuth 2.0 protocol ourselves, but rather rely on the implementation offered by GitLab, which we assume is reasonably well tested and robust. It is generally a good idea to rely on trustworthy authentication providers instead of implementing a complex protocol like OAuth 2.0 from scratch on your own.

1. Let's start by creating a Spring Boot application with relevant dependencies for Spring Security and OAuth 2.0. Use the Spring Initializr (<https://start.spring.io>) and include the dependencies `Spring Web` and `OAuth2 Client`. Then, import the project into your IDE as a Maven project

The `OAuth Client` dependency comes with batteries included, meaning that we do not explicitly need to include a Spring Security dependency on our own, as we did on the previous exercise sheet, where we used `spring-boot-starter-security`. Additionally, it enables us to make use of features and configuration options related OAuth 2.0.

2. Run the application and open your browser at `localhost:8080`. You should be redirected to `localhost:8080/login` and see a login form displayed. Remember, this is the result of Spring Security's auto-configuration we extensively discussed on the previous exercise sheet.
3. In order to use the OAuth 2.0 mechanisms of GitLab, you first need to register a *client* (see terminology in practical exercise), which in this case is going to be your Spring Boot application. To do so, navigate to <https://gitlab.lrz.de/-/profile/applications> and add a new application with at least the scopes `read_user` and `api`. For the *Redirect URI* set `http://localhost:8080/login/oauth2/code/gitlab`, this will be the callback URL that GitLab will redirect the user to, once he has successfully authorized the requesting client (assumes your Spring Boot application is running on port 8080). The result of the registration process should be a success message that looks somewhat like Figure 5.
4. Next, Spring has to be configured with the application (client) ID, the secret and the scopes to send valid authorization requests to GitLab's Authorization Server. Replace the `application.properties` with an `application.yml` (only for the purpose of readability) and add the following:

```
1 spring:
2   security:
3     oauth2:
4       client:
5         provider:
6           gitlab:
7             authorization-uri: https://gitlab.lrz.de/oauth/authorize
8             token-uri: https://gitlab.lrz.de/oauth/token
9             user-info-uri: https://gitlab.lrz.de/api/v4/user
10            user-name-attribute: username
11          registration:
12            gitlab:
13              provider: gitlab
14              redirect-uri: http://localhost:8080/login/oauth2/code/gitlab
15              authorization-grant-type: authorization_code
16              client-id: <CLIENT_ID>
17              client-secret: <CLIENT_SECRET>
```

Application: OnlineIDE

Application ID	91f6b1a4a600b9a202639 
Secret	<button>Renew secret</button>
Callback URL	http://localhost:8080/login/oauth2/code/gitlab
Confidential	Yes
Scopes	• api (Access the authenticated user's API) • read_user (Read the authenticated user's personal information)
<button>Edit</button><button>Destroy</button>	

Figure 5: LRZ GitLab Successful Application Registration

5. Finally, to demonstrate that we can use Spring Security's APIs just like we did on the previous sheet, add one controller method for the `/user` endpoint, that returns the `Principal` object, which should be your LRZ GitLab user account.

```
1 @SpringBootApplication
2 @RestController
3 public class SpringOAuth2Application {
4
5     public static void main(String[] args) {
6         SpringApplication.run(SpringOAuth2Application.class, args);
7     }
8
9     @RequestMapping("/user")
10    public Principal user() {
11        return SecurityContextHolder.getContext().getAuthentication();
12    }
13 }
```

Then, restart your application and navigate to `http://localhost:8080`. You should be redirected to LRZ GitLab and if you are not logged in already, it should show you the login form there. After a successful login, GitLab will ask for your permission to grant authorization to the client (your application) for the defined scopes. Once you have accepted these scopes, LRZ GitLab will redirect you to your own application running at `http://localhost:8080`. Now, try to access `http://localhost:8080/user` and the browser should display something like the following:

```
1 {
2   "authorities": [...],
3   "details": {...},
4   "authenticated": true,
5   "userAuthentication": {...},
6   "clientOnly": false,
7   "credentials": "",
8   "principal": "fleinen",
9   "oauth2Request": {...},
10  "name": "fleinen"
11 }
```

This concludes the first step of integrating OAuth 2.0 and Spring Security, as you now know how to use LRZ GitLab's authorization server to authenticate users and access their details through the Spring Security context.

Exercise 5: Securing an Angular Application with Spring Security

In the OnlineIDE, users will only interact with your microservices through the frontend that you will build, which includes features such as code editor, console compilation output, or project source file management. Importantly, access to these features is only granted to users who have authenticated with the GitLab LRZ (i.e. they are actually members of TUM) and are authorized to see requested resources.

Therefore, we will have to extend the previous Spring Boot application by an Angular SPA which reflects the server-side state of authentication, i.e. if the user has authenticated himself with OAuth 2.0 via GitLab LRZ. The process will not be dramatically different from the authentication process with CSRF Protection for form login on JavaScript clients introduced on the previous exercise sheet, but still needs a few noticeable tweaks.

Hence, in this week's final exercise, you will integrate your Spring Boot application that uses OAuth 2.0 with an Angular application.

1. We start with a slightly simplified version of the final Angular application from exercise sheet 7. Go ahead and open `exercise_10/spring-angular-oauth2` from your local copy of the ASE repository in an IDE.

2. Run the Spring Boot application, that is serving the Angular application.

Note: There are various ways to run the Spring Boot and the Angular application for development purposes. Two other options (than the one used here) are described in the `frontend/README.md` file.

3. Navigate to `localhost:8080`, you should see a simple navigation bar with Login and Logout button and two links to an editor and a project list view.

There is a small difference to the application from exercise sheet 7, which is that the project list is actually retrieved from the Spring Boot application (i.e. the backend). To obtain this list via the HTTP REST API from the backend at `/api/project`, we make use of the `HttpClient` service offered by Angular¹². See how it is injected and used in `app/project-list/project-list.component.ts`.

4. As you might have realized, there is no authentication mechanism in place yet, but we will set it up in the following. Start by updating the `WebSecurityConfiguration` class with a few configurations:

```
1  @EnableWebSecurity
2  @Configuration
3  public class WebSecurityConfiguration {
4
5      @Bean
6      public SecurityFilterChain configure(HttpSecurity http) throws Exception {
7          return http
8              .authorizeHttpRequests(auth -> {
9                  auth.requestMatchers("/api/**").authenticated();
10                 auth.requestMatchers("/**").permitAll();
11             })
12              .formLogin(Customizer.withDefaults())
13              .build();
14      }
15  }
```

The `@EnableWebSecurity` annotation will by default create a `/login` route to handle the login with the OAuth 2.0 authorization server. We add some boilerplate code for the `/logout`, which basically invalidates the authentication state stored in the server. Additionally, we ensure routes behind `/api` are secured, whereas `/authenticated` is a publicly available route.

In order to make this work, adapt the `application.yml` to contain the same configuration as in exercise 4 (i.e. set up OAuth 2.0 client for GitLab LRZ).

5. Next, let us set up the missing `/authentication` route defined in the configuration (here for simplicity in the main class of the application):

¹²HttpClient in Angular: <https://angular.io/guide/http>

```

1 @SpringBootApplication
2 @RestController
3 public class SpringAngularOauth2Application {
4
5     // ...
6
7     @GetMapping("/api/authenticated")
8     public boolean authenticated() {
9         SecurityContext securityContext = SecurityContextHolder.getContext();
10        Authentication authentication = securityContext.getAuthentication();
11        if (authentication != null) {
12            return authentication.getAuthorities().stream()
13                .noneMatch(grantedAuthority -> grantedAuthority.getAuthority().equals("
14                    ROLE_ANONYMOUS"));
15        }
16        return false;
17    }
18 }

```

As you can easily see, the `/api/authentication` endpoint returns *true* if the current authentication object exists and is not an anonymous user.

This endpoint allows clients (e.g. SPAs) to query their current authentication status, which will then determine whether they will be redirected to GitLab to log in again or they can use secured features as they are already logged in (e.g. access their project source files).

- Now, when you restart the application and navigate to the projects view in the frontend, you will not see the project list anymore. It will require proper authentication to view them – remember, we defined everything behind `/api/project` only accessible to users that have logged in. Yet, the frontend needs to be integrated into the authentication process, the two buttons `login` and `logout` have no effect so far.

Go ahead and add an `AuthService` and `AuthGuard` to the Angular application by running the following in frontend:

```

$ ng generate service auth
$ ng generate guard auth --implements=CanActivate

```

- The `AuthService` will contain an observable¹³, which are first class citizens in Angular¹⁴, holding a boolean value representing the current authentication state of the user in the backend. This state is requested via an HTTP GET request to the backend route `/api/authenticated` on initialization of the application (i.e. in `AuthService`'s constructor):

```

1 const LOGIN_PATH = '/login';
2 const LOGOUT_PATH = '/logout';
3
4 @Injectable({
5     providedIn: 'root',
6 })
7 export class AuthService {
8
9     private authenticated$: ReplaySubject<boolean> = new ReplaySubject<boolean>(1);
10
11     public get authenticated(): ReplaySubject<boolean> {
12         return this.authenticated$;
13     }
14
15     constructor(private httpClient: HttpClient,
16                 private router: Router) {
17         this.checkAuthentication();
18     }
19
20     public checkAuthentication(): void {
21         this.httpClient.get<boolean>('/api/authenticated')

```

¹³Observer Pattern: https://en.wikipedia.org/wiki/Observer_pattern

¹⁴Observables in Angular: <https://angular.io/guide/observables>

```

22     .subscribe((authenticated) => {
23         this.authenticated$.next(authenticated);
24     }, (err) => {
25         this.authenticated$.next(false);
26     });
27 }
28
29 public login(): void {
30     window.location.href = `${window.location.origin}${LOGIN_PATH}`; // handled by
        backend
31 }
32
33 public logout(): void {
34     this.httpClient.post(LOGOUT_PATH, {})
35     .subscribe(() => {
36         this.router.navigateByUrl('/');
37         this.checkAuthentication();
38     });
39 }
40 }

```

This allows other components of the Angular application to simply subscribe to the authentication state stored in the AuthService and react to changes in the state.

- The AuthGuard subscribes to the AuthService.authenticated observable and its canActivate method is triggered before the activation of each route which has this guard defined:

```

1  @Injectable({
2      providedIn: 'root',
3  })
4  export class AuthGuard implements CanActivate {
5
6      constructor(private authService: AuthService) {
7      }
8
9      /**
10       * Method which is activated before each route change to a route which requires this
       guard.
11       * E.g. [{ path: 'secure', component: SecureComponent, canActivate: [AuthGuard]}, ...]
12       *
13       * @param next
14       * @param state
15       */
16      canActivate(
17          next: ActivatedRouteSnapshot,
18          state: RouterStateSnapshot): Observable<boolean> {
19          return this.authService.authenticated
20             // pipe each value that is emitted by the observable
21             .pipe(
22                 // tap will intercept each emission of a value and perform some action
23                 tap(authenticated => {
24                     if (!authenticated) {
25                         // if user is not authenticated, try login via `AuthService`
26                         this.authService.login();
27                     }
28                 }));
29      }
30  }

```

- Finally, update the routes in app-routing.module.ts to require the AuthGuard before their activation

```

1  const routes: Routes = [
2      {path: 'editor', component: EditorComponent, canActivate: [AuthGuard]},
3      {path: 'projects', component: ProjectListComponent, canActivate: [AuthGuard]},
4      {path: '**', redirectTo: '/'}
5  ];

```

and add simple click event handler to the login and logout buttons in `app.component.html` (you will have to inject `AuthService` in the constructor of `app.component.ts`):

```
1 <button *ngIf="!(authService.authenticated | async)" (click)="authService.login()">Login
   </button>
2 |
3 <button *ngIf="authService.authenticated | async" (click)="authService.logout()">
   Logout</button>
```

If you are curious what the `| async` exactly means, read about pipes in Angular¹⁵ and specifically the `AsyncPipe`¹⁶.

7. Test the whole authentication process and ascertain that you are able to (i) log in (via GitLab) and out in your Angular application, (ii) can see the secured list of projects and are redirected to GitLab if you are trying to access the secured pages (editor or project list) without being logged in. Follow the exact sequence of HTTP calls with respective headers and cookies in the browser developer tools. You will notice, that Angular by default uses the CSRF protection strategy outlined in exercise sheet 9.

¹⁵Pipes in Angular: <https://angular.io/guide/pipes>

¹⁶AsyncPipe: <https://angular.io/api/common/AsyncPipe>